

LoopHole: An In-Progress Verification-Driven Lifting Pipeline for Raising Affine MLIR Loop Nests to Linalg and StableHLO

Yash Sharma

LoopHole Project (POC Milestone Report)

Abstract—This paper reports the current milestone status of LoopHole, an in-progress program-lifting pipeline that raises scalar loop nests from MLIR Affine form to higher-level tensor dialects. The present proof-of-concept (POC) integrates deterministic loop extraction, symbolic candidate ranking, SMT-based equivalence checking, and dialect-specific emission for Linalg and StableHLO targets. The contribution of this report is intentionally scoped: it is not a claim of a completed production lifter, but a conservative implementation snapshot with traceable evidence. We document architecture, command-line workflow, acceptance semantics, and current operation coverage as implemented in the repository. We also summarize artifact-level evidence from integration tests and a checked-in StableHLO batch report, then discuss constraints and active refinement areas. The objective is to provide professors and reviewers with an honest baseline that can be incrementally updated as bug fixes and coverage extensions are completed.

I. INTRODUCTION

Legacy scientific and systems kernels are often expressed as nested scalar loops. While these loops encode tensor semantics, the intent is implicit and difficult to exploit directly for modern accelerator-oriented compilation. LoopHole addresses this by lifting affine loop-based MLIR into higher-level tensor operations.

This paper is intentionally framed as an in-progress POC report. The core lifting architecture is stable at a high level, while implementation details, bug fixes, and coverage expansion are ongoing. The goal is to document what is currently implemented and verifiable in the repository, and to separate that from future roadmap items.

In practical terms, this project sits at the boundary between compiler engineering and formal verification. The engineering side focuses on robust parsing, candidate generation, and correct dialect emission. The verification side focuses on checking whether a candidate high-level sketch is semantically equivalent to the source loop behavior. The interaction between these two sides is important: a strong symbolic guess without proof should not be treated as the same confidence level as a proved equivalence result.

A. Problem framing

Many kernels of interest (for example matrix multiplication, transpose, and reductions) are represented as scalarized loops in legacy code paths. Even when the loop is affine and regular, recognizing the intended tensor operation is non-trivial if a tool only sees low-level memory operations. A lifting system must

infer operator shape and semantics from access patterns and arithmetic structure while preserving correctness.

B. What this paper claims

This report makes only three classes of claims:

- 1) **Implemented now**: behavior that is directly supported by current source code.
- 2) **Observed now**: outcomes demonstrated by tests and checked-in artifacts.
- 3) **Planned next**: roadmap items explicitly marked as future work.

This claim discipline is deliberate because the project is still under active development.

II. DESIGN GOALS AND NON-GOALS

The current POC is guided by a small set of explicit design goals.

A. Primary goals

- **Conservative correctness posture**: distinguish formally proved outputs from unresolved or refuted outcomes.
- **Transparent implementation**: keep the lifting stages modular and inspectable.
- **Traceable evidence**: ensure important claims can be mapped to code, tests, or report artifacts.
- **Practical usability**: provide a command-line workflow for single-file and batch runs.

B. Current non-goals

- Full operator completeness across all MLIR or StableHLO op families.
- Broad benchmark-scale performance claims.
- Production-grade robustness guarantees for every malformed input pattern.

These non-goals prevent over-claiming and keep the scope aligned with the current milestone.

III. CURRENT SYSTEM ARCHITECTURE

The current pipeline can be summarized as:

Affine MLIR → Parse → SymPy Match
→ Z3 Check → Linalg or StableHLO Emit (1)

Operationally, this corresponds to the following stages in the codebase:

- 1) **Parse and extract loop semantics:** induction variables, memory access structure, and function metadata.
- 2) **Symbolic candidate ranking:** a bounded sketch library is scored against extracted structure.
- 3) **SMT verification:** top candidates are checked with Z3 for equivalence or non-equivalence outcomes.
- 4) **Dialect emission:** matching sketches are emitted as Linalg or StableHLO MLIR.

This staged design provides explicit checkpoints and allows conservative acceptance policies where formally proved results are distinguished from unresolved or refuted cases.

A. Stage 1: Parsing and extraction

The parser extracts loop-level structure, induction variables, tensor names, access indices, and arithmetic operations from affine-style MLIR text. This step also produces diagnostics used by downstream stages and by the CLI. A reliable extraction layer is critical because all later reasoning depends on this representation.

B. Stage 2: Symbolic candidate ranking

The symbolic phase scores a bounded sketch library against extracted behavior. The top- k candidates are forwarded to verification. This design allows flexible ranking heuristics while maintaining a deterministic verification checkpoint.

C. Stage 3: SMT verification

Candidate sketches are checked with Z3 under timeout constraints. At a high level, the checker attempts to separate equivalent mappings from refuted mappings. The system then records a result state that can be used in strict or non-strict acceptance mode.

D. Stage 4: Emission and artifact checks

After candidate selection, the emitter generates target-dialect MLIR and can optionally integrate external verifier tooling for artifact-level validation. This keeps the emitted output as a first-class artifact rather than only an internal string.

IV. IMPLEMENTATION SNAPSHOT

This section summarizes implemented behavior in the current repository snapshot.

A. CLI workflow and operational surface

The CLI supports both single-input and batch-oriented operation. Representative command families include:

- source-to-MLIR conversion pathways
- direct lifting pathways
- verification-focused pathways
- batch reporting pathways
- demo and sketch-inspection utilities

In practice this makes it possible to run exploratory local workflows as well as stricter, report-oriented workflows in a reproducible manner.

TABLE I
CURRENT ACCEPTANCE SEMANTICS IN THE POC

State	Accepted in strict mode	Interpretation
Proved	Yes	Candidate is solver-verified equivalent
Unproved timeout or unknown	No	Candidate not refuted, but not fully proved
Refuted	No	Candidate contradicted by solver checks

B. Acceptance semantics

The implementation uses explicit result-state semantics. A simplified view is shown in Table I.

This separation is important for an in-progress project because it avoids conflating high confidence with formally proved.

C. Bounded operator coverage

Current lifting coverage is finite and hand-authored through sketch definitions and corresponding emitters. Supported families include:

- Linalg-oriented kernels: matmul, matvec, vecmat, dot, transpose, copy, elementwise arithmetic, selected reductions, and selected convolution and pooling variants.
- StableHLO-oriented kernels: dot_general variants, transpose, add, subtract, multiply, reduce sum, reduce max, and convolution.

This should be interpreted as bounded POC capability rather than complete operator coverage.

V. EVALUATION PROTOCOL IN THIS MILESTONE

This milestone uses a practical artifact-driven evaluation protocol instead of a full benchmark campaign.

A. Data sources used for evidence

Evidence in this draft is drawn from:

- integration tests under the project test tree
- a checked-in StableHLO batch report artifact
- emitted lifted MLIR artifacts generated during batch workflows

B. Reported metrics

The report path currently emphasizes reliability-oriented metrics:

- number of processed fixtures
- proved versus refuted versus unresolved counts
- per-fixture timing summaries
- emitted output presence for accepted candidates

This metric profile is appropriate for a verification-driven POC where semantic trust state matters more than throughput.

TABLE II
CURRENT EVIDENCE SNAPSHOT FROM CHECKED-IN BATCH REPORT
ARTIFACT

Metric	Value
Total processed fixtures	7
Formally proved	5
Refuted	2
Unproved timeout	0

TABLE III
REPRESENTATIVE FIXTURE OUTCOMES IN THE CHECKED-IN STABLEHLO
REPORT

Fixture	Outcome	Notes
matmul.mlir	Proved	dot_general mapping
transpose.mlir	Proved	transpose mapping
dot_product.mlir	Proved	vector dot mapping
elementwise_add.mlir	Proved	elementwise add mapping
reduce_sum.mlir	Proved	reduce add mapping
conv1d.mlir	Refuted	no verified candidate
conv2d.mlir	Refuted	no verified candidate

VI. EVIDENCE SNAPSHOT FROM REPOSITORY ARTIFACTS

The repository contains a checked-in StableHLO batch report artifact with the following summary: total files = 7, proved = 5, refuted = 2, unresolved-timeout = 0. In this snapshot, refuted examples include conv1d and conv2d fixtures, while several dense linear-algebra-style kernels are proved.

Integration tests in the repository further exercise core paths including stablehlo operation coverage checks, batch report schema behavior, demo smoke flow, and matmul integration behavior.

A. Per-fixture summary view

To make the artifact-level evidence more concrete, Table III summarizes fixture outcomes that are represented in the checked-in batch report.

The key interpretation is that the architecture is functioning end-to-end on a bounded subset, while specific convolution paths still require additional refinement in this snapshot.

VII. END-TO-END OPERATIONAL WALKTHROUGH

To clarify how this system behaves in practice, this section describes a typical run from input function to report outcome.

A. Step A: Input and extraction

The workflow starts from an affine-style MLIR function. The extractor identifies loop order, index variables, tensor accesses, and arithmetic forms that define the computation. At this stage, no high-level tensor operator is assumed. The objective is to preserve source behavior in a normalized internal representation.

TABLE IV
CLAIM-TO-EVIDENCE TRACEABILITY MATRIX USED IN THIS REPORT

Claim category	Evidence type	Example in this milestone
Implemented behavior	Source modules	parser, lifter, sketch, verifier, emitter pipeline
Observed outcomes	Test/report artifacts	integration tests and StableHLO batch report summary
Roadmap statements	Explicitly labeled future work	bug-fix and coverage-expansion items for next pass

B. Step B: Candidate generation

The symbolic ranking stage compares extracted behavior to known sketch templates. Instead of attempting unbounded synthesis, the current POC uses a finite candidate set with ranking and top- k filtering. This keeps latency predictable and simplifies debugging when a case is not matched or is weakly matched.

C. Step C: Verification and trust-state assignment

Candidate(s) then enter the SMT check stage. The result is not just a boolean success flag; it is a trust-state assignment used by downstream policy:

- **Proved:** formally accepted in both strict and non-strict modes.
- **Unproved timeout or unknown:** may be emitted in exploratory modes but is not accepted in strict mode.
- **Refuted:** rejected.

This policy is central to the project philosophy because it preserves a clean boundary between heuristic confidence and formal correctness.

D. Step D: Emission, reporting, and triage

Once a candidate is accepted under the selected policy, the emitter generates target MLIR. In batch workflows, result records are aggregated into JSON reports that include status counts, per-file details, and timing metadata. These report artifacts are intentionally designed to support triage and milestone comparisons over time.

E. Observed behavior in current milestone

For the current artifact snapshot, dense linear-algebra examples are generally represented among proved outcomes, while selected convolution fixtures remain refuted. This behavior is consistent with the bounded sketch coverage and active refinement areas documented elsewhere in this paper.

VIII. CLAIM-TO-EVIDENCE TRACEABILITY

Because this is a milestone report, every strong claim should map to explicit evidence. Table IV summarizes this mapping style.

This matrix is intentionally simple, but it helps prevent scope drift when the manuscript is updated across iterations.

IX. DISCUSSION

A. Why conservative reporting is useful

In an active project, reporting only aggregate success percentages can obscure what is genuinely proved. The current state model helps avoid that ambiguity by exposing which cases are solved, unresolved, or contradicted.

B. Where current engineering effort is concentrated

Based on present artifacts and tests, the immediate engineering pressure points include:

- reducing mismatch cases in convolution-related lifting paths
- improving robustness of parser and shape/type metadata handling
- expanding coverage while preserving strict verification semantics

C. Implications for a next milestone

A near-term milestone can be evaluated by whether it shifts refuted or unresolved cases into proved outcomes without relaxing correctness policy. This is more meaningful than only increasing emitted-output counts.

X. MODULE-LEVEL TECHNICAL NOTES

This section provides additional implementation detail for readers who want to understand how the current POC behaves internally. The intent is to make this report more useful as a technical handoff document, not just a high-level summary.

A. Affine extraction module behavior

The extraction stage is designed to convert textual affine-style MLIR into a structured form that downstream modules can consume reliably. At this stage, the implementation attempts to identify loop variables, loop bounds, tensor access expressions, and arithmetic operations in a deterministic way. If extraction fails or becomes ambiguous, the pipeline records diagnostics early rather than silently inferring unsupported behavior. This conservative parser posture is important for maintaining correctness trust in later verification stages.

B. Sketch library structure

The sketch library is currently a hand-authored catalogue of candidate tensor semantics. In practical terms, each sketch represents an expected relationship among loop indices, input tensors, output tensors, and compute payload forms (for example accumulate multiply, elementwise add, or reductions). The strength of this approach is interpretability and direct control over supported patterns. The limitation is coverage: behavior outside known sketch patterns cannot currently be lifted as proved output.

C. Symbolic ranking strategy

The symbolic stage evaluates extracted behavior against the sketch catalogue and assigns ranking scores. This stage is intentionally separated from formal verification so that candidate generation can evolve independently. In the current milestone, this separation has two engineering advantages. First, debugging is clearer because symbolic mismatches can be analyzed before solver interaction. Second, top- k candidate filtering keeps verification cost bounded for exploratory and batch workflows.

D. SMT checker behavior and policy relevance

The SMT stage determines whether a candidate should be treated as equivalent, unresolved, or refuted under current model assumptions and timeout limits. The result is then surfaced as part of a policy-visible state machine used by strict and non-strict workflows. This design ensures policy choices are explicit. For example, strict mode can reject unresolved outputs even when symbolic confidence appears high, which prevents accidental elevation of unproven candidates in professor-facing or report-facing outputs.

E. Emitter behavior for target dialects

Once a candidate is accepted under policy, emission converts it to target MLIR syntax for either Linalg or StableHLO pathways. Emitter constraints are intentionally explicit: required metadata must be present, and unsupported mappings should fail loudly rather than produce ambiguous output. This is consistent with the project’s conservative philosophy. The emitted artifact can then be inspected directly or fed to optional external validation tooling.

F. CLI and reporting integration

The command-line layer integrates parser, matcher, checker, and emitter stages for both single-run and batch workflows. In batch mode, the reporting path writes structured JSON with aggregate counts and per-file outcomes. This report structure is useful for milestone comparison because it preserves not only success counts but also refutation and unresolved states. As the project advances, keeping this schema stable will make it easier to show quality movement between revisions.

G. Current reliability lessons

A practical lesson from this milestone is that verification-driven lifting quality depends as much on metadata fidelity and extraction reliability as on solver strength. In other words, solver-backed equivalence alone does not solve pipeline reliability if upstream extraction, shape modeling, or candidate selection are inconsistent. For that reason, future work in this project should continue balancing algorithmic expansion with engineering hardening.

H. How this section should be updated later

When the next pass of bug fixes is completed, this module-level section should be revised with concrete deltas:

- which module-level changes were made
- which previously refuted or unresolved fixtures changed state
- whether strict-mode acceptance coverage increased
- whether diagnostics improved for failed cases

Maintaining this level of explicitness will support stronger publication readiness over time.

XI. THREATS TO VALIDITY

This paper reports an implementation snapshot and therefore has clear validity limits.

A. Internal validity

The current fixture set is bounded. Outcomes may change as parser and sketch logic evolve. Strict mode helps reduce false confidence, but it does not eliminate modeling assumptions.

B. External validity

The checked-in report does not represent full benchmark-suite coverage. Therefore, conclusions in this draft should not be interpreted as claims about broad workload generalization.

C. Construct validity

The current metrics focus on trust-state outcomes and not end-to-end runtime speedups on hardware backends. That choice is intentional for this stage but limits performance-oriented conclusions.

XII. LIMITATIONS AND IN-PROGRESS AREAS

This project is not complete. The current implementation is a milestone toward a broader system, and the following constraints remain material:

- Coverage is limited to the current sketch/emitter set and does not yet represent broad, unconstrained lifting.
- Some high-level project documents discuss wider benchmark and operator targets that should be treated as roadmap, not current implementation evidence.
- Internal robustness and bug-fix work is ongoing and expected to improve both reliability and coverage in later passes.

Accordingly, this paper avoids claiming comprehensive benchmark-scale performance or full operator completeness.

XIII. ROADMAP-ALIGNED NEXT STEPS

The next revision cycle is expected to preserve the high-level architecture while improving internal reliability and coverage. Concretely, high-priority next steps are:

- resolve known bug classes in candidate selection and emission pathways
- expand verified fixture coverage before adding broader benchmark claims
- improve diagnostics so failed or refuted cases are easier to triage

- keep result reporting schema stable to simplify milestone-to-milestone comparison

This staged plan enables incremental progress without weakening claim quality.

XIV. CONCLUSION

LoopHole currently demonstrates a working verification-driven lifting architecture from Affine MLIR loops to Linalg or StableHLO outputs, with concrete test and report evidence for a bounded kernel subset. The present contribution is a conservative, implementation-aligned milestone report: the high-level lifting and SymPy/Z3 verification strategy is stable, while internal fixes and broader coverage remain active engineering work. A subsequent revision will update quantitative scope and operator breadth after the next implementation pass.

REPRODUCIBILITY NOTE

This draft is aligned to the current repository snapshot and to a separate findings file maintained under `rpaper/docs/implementation_findings.md`. Future revisions should update that findings file first, then synchronize the manuscript claims.